Solution Design Document

Stop-Loss and Profit-Target Exits for a Cost-Losing Intraday Reversal Rule | Ejike, Odeajo, Kalu

1. Business problem

Traders often add a stop-loss or a profit target to short-term trades, assuming it protects them. Backtests built on one-minute price bars cannot see which price came first inside a minute, and many ignore trading costs. A desk can therefore adopt exit rules that look safe on paper but lose money in practice.

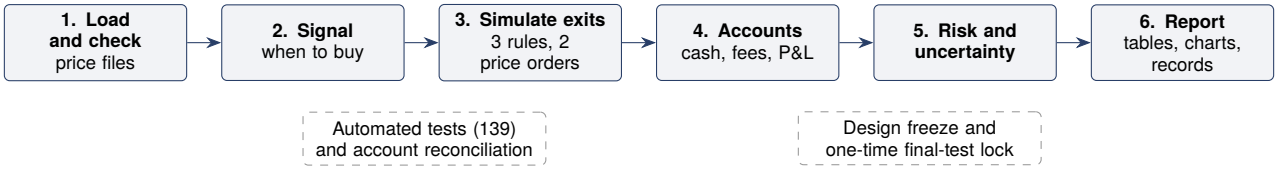
2. What the solution does

It replays three years of trading days (2021 to 2023) for five large Indian stocks and applies one fixed buying rule. Every trade is then closed three ways, after 15 minutes, at a stop-loss, or at a stop-loss or profit target, with identical entries and sizes. The only difference between the three results is the exit rule. Costs, delays and both possible price orders inside a minute are tested explicitly.

3. Specification

Inputs	Instructor minute price files (stocks and NIFTY 50 index), read from Google Drive or a local folder without modification; one declared supplement restores a broken day.
Outputs	Tables and charts of average daily profit, worst-day losses (expected shortfall), account drawdown, cost breakdown and sensitivity; a run record with fingerprints of code and data.
Fixed settings	Five stocks, one buying rule, 15-minute holding limit, 15 bp trading cost, INR 100,000 reference capital, 60/20/20 split by date.
Success criteria	Same inputs give the same outputs; every account balances to within INR 0.0000001; the final test period can be opened only once, after the design is frozen.
Platform	Python on Google Colab (no installation needed) or locally in VS Code or Jupyter; one notebook runs the whole study.

4. How it works



5. Where each part lives and what it achieves

Component	Location in code	What it achieves
1. Load and check	Notebook: *Load instructor data*; `src/path_robust/vendor.py`	Reads only the needed files, rejects malformed rows, never fills gaps, applies one declared and checked supplement, records what was used.
2. Signal	Notebook: *Signal and sizing*	Flags a stock that fell unusually far behind the index, using past data only.
3. Simulate exits	Notebook: *Execution and account balances*; `src/path_robust/engine.py`	Closes each trade under all three rules and both possible price orders, with costs and slippage.
4. Accounts	Notebook: *Session replay and funded segments*	Tracks cash daily and as one continuous account; stops if money would run out.
5. Risk and uncertainty	Notebook: *Paired uncertainty and tail risk*	Measures worst-day losses, drawdown and how sure we can be about each difference.
6. Report	Notebook: *Save the run and review the freeze*; `reports/m7_final/make_tables.py`	Saves each run privately, then publishes a reviewed run to the repository with checksums; redraws the report's charts from the saved results.
Controls	`tests/`; notebook freeze cell	Prove calculations on known examples; prevent peeking at or rerunning the final test.

6. Risks and limits accepted for sign-off

Results describe five stocks and one rule, not the whole market. Fills, spreads and the data's time-stamp convention are assumptions, tested both ways where possible. The tool evaluates exit rules; it does not place real trades. Outcome: in all three periods the buying rule earned less than its costs, and adding a stop and target lowered average return while reducing the worst days slightly; data gaps cut the final test to 88 of 148 days.

Code and instructions: `https://github.com/I-am-Uchenna/nse-intraday-risk-controls`

Approved by: _____ Date: _____